

Piecewise PV Inversion Handbook

The Wu Fortran Reference (Benchmark) and the Python Spherical-Harmonic Port

For the 2025-01-08 California Blocking Case Study

x_yan
pv_inversion/

June 3, 2026

Contents

1	Introduction and Overview	5
1.1	Purpose of this Handbook	5
1.2	Notation Cheat-Sheet	6
1.3	Quick Tour of the Code	6
I	Theory and the Wu Fortran Reference (Benchmark)	7
2	Mathematical Foundations	8
2.1	Coordinates and Constants	8
2.2	Thermodynamic Relations	8
2.3	Non-uniform Vertical Finite Differences	9
2.4	Ertel Potential Vorticity	9
2.5	Linearization about a Mean State	9
2.6	Closure: Coupling $\delta\Phi$ to $\delta\psi$	10
3	The Wu Fortran Reference Solver	11
3.1	Grid and Vertical Levels	11
3.2	The Three Programs and Four Passes	11
3.3	Solver Parameters	12
4	The Wu Pipeline, Step by Step (Benchmark)	13
4.1	Step 05 — Pass A/B: Mean and Event PV, Balanced ψ	13
4.2	Step 06 — Pass C: Total Balanced Inversion (BALNC)	14
4.3	Step 07 — Pass D: Piecewise Perturbation Inversion (BALP)	15
4.4	Step 08 — Parse Outputs and ERA5 Ertel PV	16
4.5	Step 09 — PV Advection by the Induced Winds	17
4.6	Step 10 — Fig. 8 Replica (the Benchmark Result)	18
4.7	SI Output Convention (adopted 2026-06-03)	20
II	Wu Python Finite-Difference Port	21
5	Finite-Difference Operators	22
5.1	Grid and Map Factors	22
5.2	Five-Point Laplacian	23
5.3	Poisson Solver (Red-Black SOR)	23
5.4	Implicit Vertical Tridiagonal Solver	23

5.5	Summary: FD vs SH Operators	24
6	Forward PV Computation	25
6.1	Ertel PV on Sigma Levels	25
6.2	Balanced Streamfunction	25
6.3	Boundary Streamfunction from Wind Integration	25
6.4	Implementation	26
7	The Wu Non-Dimensional BALNC Port (Spherical Harmonics)	27
7.1	Wu Non-Dimensionalisation	27
7.2	Vertical Operator: BB, BH, BL	28
7.3	The Coupled BALNC System	29
7.4	SH Realisation of the Wu Operators	29
7.5	Discrepancies between the Wu FD Reference and the SH Port	30
7.6	Open Issues / Not Yet Done	30
8	Conclusions and Outlook	32
8.1	Summary	32
8.2	Status and Known Limitations	32
8.3	Next Steps	33
A	Notation Glossary	34
B	Public-API Reference	35
B.1	Part I — Wu Fortran reference (wu/)	35
B.1.1	config.py (shared)	35
B.1.2	Fortran programs (wu/fortran/)	35
B.1.3	Step drivers (wu/steps/)	35
B.2	Part II — Wu Python finite-difference port (wu_python/)	35
B.2.1	wu_python/core/grid.py	35
B.2.2	wu_python/core/nondim.py	36
B.2.3	wu_python/core/fd_ops.py	36
B.2.4	wu_python/core/sor_solver.py	36
B.2.5	wu_python/core/pv_calc.py	36
B.2.6	wu_python/core/balance.py (BALNC)	36
B.2.7	wu_python/core/piecewise.py (BALP)	36
B.2.8	wu_python/config.py	36
B.2.9	Step drivers (wu_python/steps/)	37
B.3	Shared utilities (pvtend.sh_ops)	37

List of Figures

4.1	Pass A mean Ertel PV	13
4.2	Event PV anomaly	14
4.3	Balanced mean streamfunction	14
4.4	Pass C refinement	15
4.5	Pass C all levels	15
4.6	Pass D three pieces	16
4.7	Piece 3 small-scale noise	16
4.8	Wu Q vs ERA5 PV	17
4.9	PV advection per piece	18
4.10	Davis 2022 Fig. 8 replica	19

List of Tables

5.1	Operator comparison: Wu Python (FD) vs retired SH port	24
-----	--	----

Chapter 1

Introduction and Overview

1.1 Purpose of this Handbook

This handbook documents **piecewise potential-vorticity inversion** (PPVI) for atmospheric blocking, as implemented in `pv_inversion/`. The goal of the project is to reproduce the diagnostic of Davis et al. (2022, *J. Climate*, their Fig. 8) for the 2025-01-08 California blocking event: partition the upper-level PV anomaly into vertical pieces, invert each piece to a balanced circulation, and quantify how each piece advects PV to sustain the block.

The repository carries **two independent inversion tracks** that share a common ERA5 data-preparation front end:

1. the **Wu Fortran reference** (`pv_inversion/wu/`) — W. Wu’s finite-difference successive-over-relaxation (SOR) solver (`pvpialln_94UV.f`, `qinvert21_94.f`, `qinvertp21_94.f`). This track **works**: it reproduces the Davis 2022 Fig. 8 diagnostic end-to-end and is therefore treated as the *benchmark*. It is the subject of **Part I** (Chapters 3 and 4), where every pass is illustrated with the plot it generates.
2. the **Wu Python finite-difference (FD) port** (`pv_inversion/wu_python/`) — a faithful re-implementation of Wu’s non-dimensional balance system on the same regional 51×87 lat-lon grid, using finite differences and numba-accelerated SOR. This is the subject of **Part II** (Chapters 5 to 7). It documents the operator mapping, forward PV computation, non-dimensionalization, and the current state of the pure-Python pipeline.

What was retired

Three earlier solvers have been removed from the codebase and from this handbook:

- Two SH solvers — a one-step *Direct Spectral Solve* and an *LGMRES + linearized-Charney* solver (collectively “Family-B”). Their governing closure (the linear f_0 -plane balance) is retained in Section 2.6.
- The **Python spherical-harmonic (SH) BALNC port** — this was a faithful re-implementation on the full-hemisphere SH grid using `pvtend.sh_ops`, but it did not converge and has been superseded by the Wu Python finite-difference port (`wu_python/`).

1.2 Notation Cheat-Sheet

Symbol	Meaning
ψ, Φ	streamfunction ($\text{m}^2 \text{s}^{-1}$), geopotential ($\text{m}^2 \text{s}^{-2}$)
$\bar{\psi}, \bar{\Phi}$	climatological mean (basic) state
$\delta\psi = \psi - \bar{\psi}, \delta\Phi = \Phi - \bar{\Phi}$	anomalies
q	Ertel potential vorticity ($\text{PVU} = 10^{-6} \text{K m}^2 \text{kg}^{-1} \text{s}^{-1}$)
$\bar{q}, q' = q - \bar{q}$	mean and anomalous PV
$\zeta = \nabla^2 \psi$	relative vorticity
$f = 2\Omega \sin \phi, f_0 = f(45^\circ)$	Coriolis parameter, reference value
$\pi = (p/p_0)^\kappa$	Exner function ($p_0 = 10^5 \text{Pa}, \kappa = R/c_p$)
$\theta = T(p_0/p)^\kappa$	potential temperature
$A_{\text{avo}} = f + \bar{\zeta}, B_{\text{stb}} = \partial^2 \bar{\Phi} / \partial \pi^2$	avorticity, static stability proxy
∇^2, ∇^{-2}	horizontal Laplacian and its spectral inverse
D_1, D_2	first/second-derivative matrices in π

A more complete glossary appears in [Appendix A](#).

1.3 Quick Tour of the Code

Part I — the Wu Fortran reference (`pv_inversion/wu/`).

Component	Purpose	Reference
<code>wu/fortran/pvpialln_94UV.f</code>	Pass A/B: Ertel PV + balanced ψ, Φ	Chapter 3
<code>wu/fortran/qinvert21_94.f</code>	Pass C: total balanced inversion (BALNC)	Chapter 3
<code>wu/fortran/qinvertp21_94.f</code>	Pass D: piecewise perturbation (BALP)	Chapter 3
<code>wu/steps/05...10/</code>	the step-by-step pipeline + plots	Chapter 4
<code>config.py</code>	grid, levels, solver parameters	Chapter 3

Part II — the Python SH port (`pv_inversion/sh/`).

Module	Purpose	Reference
<code>sh_ppvi/coords.py</code>	vertical Exner grid, D_1, D_2 , Coriolis	Chapter 2
<code>pvtend.sh_ops</code>	SH gradient/Laplacian/Helmholtz (reference only)	Chapter 5
<code>wu_python/core/pv_calc.py</code>	Ertel PV diagnostic	Chapter 6
<code>sh_ppvi/nondim.py</code>	Wu non-dimensional scales	Chapter 7
<code>sh_ppvi/sor.py</code>	BB, BH, BL vertical stencil	Chapter 7
<code>sh_ppvi/wu_ops.py</code>	Wu-scaled SH operator wrappers	Chapter 7
<code>sh_ppvi/balnc.py</code>	coupled BALNC ($H-\psi$) Picard solve	Chapter 7
<code>sh_ppvi/balp.py</code>	piecewise BALP (pending convergence)	Chapter 7

Part I

Theory and the Wu Fortran Reference (Benchmark)

Chapter 2

Mathematical Foundations

2.1 Coordinates and Constants

The vertical coordinate is the **Exner pressure**

$$\pi \equiv \left(\frac{p}{p_0} \right)^\kappa, \quad p_0 = 10^5 \text{ Pa}, \quad \kappa = \frac{R_d}{c_p} = \frac{287}{1004} \approx 0.2858 \quad (2.1)$$

with $\pi = 1$ at the surface and $\pi \rightarrow 0$ at the model top. On the $K = 10$ ERA5 pressure levels used here we obtain

$$\pi_k = [1.000, 0.978, 0.955, 0.903, 0.864, 0.820, 0.770, 0.709, 0.673, 0.631]. \quad (2.2)$$

Constant	Symbol	Value
Gravity	g	9.80665 m s^{-2}
Dry-air gas constant	R_d	$287 \text{ J kg}^{-1} \text{ K}^{-1}$
Specific heat at p	c_p	$1004 \text{ J kg}^{-1} \text{ K}^{-1}$
$\kappa = R_d/c_p$	κ	0.28586
Earth's rotation	Ω	$7.292 \times 10^{-5} \text{ rad s}^{-1}$
Earth radius	a	$6.371 \times 10^6 \text{ m}$
Reference pressure	p_0	10^5 Pa
Reference Coriolis	f_0	$f(45^\circ) \approx 1.031 \times 10^{-4} \text{ rad s}^{-1}$

2.2 Thermodynamic Relations

Potential temperature (`coords.theta_from_T`):

$$\theta = T \left(\frac{p_0}{p} \right)^\kappa. \quad (2.3)$$

Hydrostatic balance in Exner coordinates (`coords.dpi_phi_from_theta`):

$$\frac{\partial \Phi}{\partial \pi} = -c_p \theta \quad (2.4)$$

which is the dry-static-energy form of hydrostatic balance. It provides the *vertical* link between Φ and θ that will close the inversion.

Coriolis (`coords.coriolis`):

$$f(\phi) = 2\Omega \sin \phi, \quad \frac{df}{dy} = \frac{2\Omega \cos \phi}{a} \equiv \beta. \quad (2.5)$$

2.3 Non-uniform Vertical Finite Differences

ERA5 levels are unevenly spaced in π ; we therefore use three-point Lagrange stencils. At an interior node π_0 with neighbours $\pi_m < \pi_0 < \pi_p$ and spacings $h_m = \pi_0 - \pi_m$, $h_p = \pi_p - \pi_0$:

$$\left. \frac{\partial f}{\partial \pi} \right|_{\pi_0} \approx -\frac{h_p}{h_m(h_m + h_p)} f_m + \frac{h_p - h_m}{h_m h_p} f_0 + \frac{h_m}{h_p(h_m + h_p)} f_p, \quad (2.6)$$

$$\left. \frac{\partial^2 f}{\partial \pi^2} \right|_{\pi_0} \approx \frac{2}{h_m(h_m + h_p)} f_m - \frac{2}{h_m h_p} f_0 + \frac{2}{h_p(h_m + h_p)} f_p. \quad (2.7)$$

At the two boundaries we substitute the second-order one-sided analogues. Assembling the stencils into a matrix gives

$$\frac{\partial f}{\partial \pi} = \mathbf{D}_1 f, \quad \frac{\partial^2 f}{\partial \pi^2} = \mathbf{D}_2 f, \quad \mathbf{D}_1, \mathbf{D}_2 \in \mathbb{R}^{K \times K}. \quad (2.8)$$

These matrices are precomputed once in `coords.D1_MAT`, `coords.D2_MAT`.

2.4 Ertel Potential Vorticity

The hydrostatic Ertel PV reads

$$Q = -g(\zeta + f) \frac{\partial \theta}{\partial p}. \quad (2.9)$$

Substituting eq. (2.4) and the chain rule $\partial/\partial p = (\kappa\pi/p) \partial/\partial \pi$ gives the (ψ, Φ) -only form used throughout the code:

Ertel PV in psi-Phi form

$$Q(\psi, \Phi) = g(\nabla^2 \psi + f) \cdot \underbrace{\frac{\kappa\pi}{p c_p}}_{\equiv \text{coeff}(p)} \cdot \frac{\partial^2 \Phi}{\partial \pi^2} \times 10^6 \text{ [PVU]} \quad (2.10)$$

The factor 10^6 converts SI units to PVU. Equation eq. (2.10) is implemented in both the Wu Python track (`wu_python/core/pv_calc.py`, Chapter 6) and the Wu Fortran subroutine `PV_CALC` (Chapter 3).

2.5 Linearization about a Mean State

Decompose $\psi = \bar{\psi} + \delta\psi$, $\Phi = \bar{\Phi} + \delta\Phi$ and Taylor-expand eq. (2.10) to first order, retaining only $\mathcal{O}(\delta)$ terms. After cancellation of the mean contribution $\bar{q} = Q(\bar{\psi}, \bar{\Phi})$:

Linearized Ertel PV

$$q' = g \cdot \text{coeff}(p) \left[\underbrace{(f + \nabla^2 \bar{\psi})}_{A_{\text{avo}}} \frac{\partial^2 \delta\Phi}{\partial \pi^2} + \underbrace{\frac{\partial^2 \bar{\Phi}}{\partial \pi^2}}_{B_{\text{stb}}} \nabla^2 \delta\psi \right] \times 10^6 \quad (2.11)$$

The two basic-state coefficients

$$A_{\text{avo}}(\mathbf{x}) = f + \nabla^2 \bar{\psi} = f + \bar{\zeta}, \quad B_{\text{stb}}(\mathbf{x}) = \frac{\partial^2 \bar{\Phi}}{\partial \pi^2} \quad (2.12)$$

have natural meteorological interpretations: A_{avo} is the *absolute vorticity* of the mean state, and B_{stb} is proportional to the mean *static stability* (via eq. (2.4), $B_{\text{stb}} = -c_p \partial \bar{\theta} / \partial \pi$).

2.6 Closure: Coupling $\delta\Phi$ to $\delta\psi$

Equation eq. (2.11) is one equation in two unknowns $(\delta\psi, \delta\Phi)$. Closure requires a **balance constraint**. Two are used in the code:

(a) **Nonlinear Charney balance** (the mean-state balance of the nonlinear Picard inversion):

$$\nabla^2 \Phi = \nabla \cdot (f \nabla \psi) + 2 J(\psi_x, \psi_y). \quad (2.13)$$

(b) **Linear f_0 -plane balance** (the linear closure referenced by the spherical-harmonic BALNC port, Chapter 7):

$$\delta\Phi \approx f_0 \delta\psi \implies \frac{\partial^2 \delta\Phi}{\partial \pi^2} \approx f_0 D_2 \delta\psi \quad (2.14)$$

Substituting eq. (2.14) into eq. (2.11) yields a linear operator that acts *only on* $\delta\psi$:

$$q' = g \text{coeff}(p) [A_{\text{avo}} f_0 D_2 \delta\psi + B_{\text{stb}} \nabla^2 \delta\psi] \times 10^6 \quad (2.15)$$

Equation eq. (2.15) was the central object of the legacy spherical-harmonic piecewise solvers (a *direct* variant dropped the first term; an *LGMRES* variant kept both). Those Family-B solvers have since been retired; the relation survives here as the linear closure cited by the SH BALNC port (Chapter 7).

Caveat of the f_0 -plane closure

$\delta\Phi \approx f_0 \delta\psi$ is the geostrophic-balance approximation at a reference latitude. It under-estimates the geopotential anomaly at upper levels by a factor that depends on the meridional structure of $\delta\psi$. This is the principal source of the $\approx 0.1 \times$ *amplitude bias* of the SH port vs. the Wu Fortran reference (Chapter 4).

Chapter 3

The Wu Fortran Reference Solver

This chapter describes the working benchmark: W. Wu’s finite-difference piecewise PV-inversion code, ported only in the sense of being *driven* from Python (`wu/steps/`) while the numerical core remains the original Fortran-77. The three programs live in `wu/fortran/` and are compiled with `gfortran -std=legacy -O2 -fno-automatic`.

3.1 Grid and Vertical Levels

The solver operates on a bounded Northern-Hemisphere window (`config.py`):

Quantity	Symbol / code	Value
Longitudes	N_X	87
Latitudes	N_Y	51
Levels	N_W	10
Latitude row	$\phi_i = 85.5 - 1.5 i$	$85.5^\circ \rightarrow -13.5^\circ$ (N→S)
Longitude col	$\lambda_j = -169.5 + 1.5 j$	$-169.5^\circ \rightarrow -40.5^\circ$
Pressure levels	PLEVS [hPa]	1000, 925, 850, 700, 600, 500, 400, 300, 250, 200
Earth radius	$R_\oplus = 2 \times 10^7 / \pi$	6.366×10^6 m
Grid spacing	$\Delta p = \Delta l = R_\oplus \text{rad}(1.5^\circ)$	1.667×10^5 m

Fortran PARAMETER blocks are hard-wired

N_X, N_Y appear as compile-time **PARAMETER**s inside every `.f` file. Changing the grid requires editing and recompiling the Fortran, not just `config.py`. The Python drivers pass solver *parameters* (relaxation factors, iteration caps) over `stdin`; they cannot change the array dimensions.

3.2 The Three Programs and Four Passes

Pass	Program	Subroutine	Produces
A	pvpialln	—	meanq, meanh (mean PV, ψ , Φ)
B	pvpialln	—	event_q.out, event_h.out (event PV, ψ , Φ)
C	qinvert21	BALNC	event_bal.out (total balanced ψ , Φ)
D	qinvert21	BALP	event_pert.out (3-piece perturbation ψ')

Pass A/B — pvpialln. Computes Ertel PV on the interior σ -levels ($K = 2 \dots N_W - 1$, i.e. 925–250 hPa), the balanced streamfunction ψ by relative-vorticity inversion, the geopotential Φ on pseudo-height, and the boundary potential temperature θ at top and bottom. Pass A is run on the 11-day climatological mean state; Pass B on the event state.

Pass C — qinvert21 (BALNC). Solves the full *total* balanced inversion: a 2-D Helmholtz equation for ψ on each σ -level coupled to a 3-D PV– ψ relation under gradient-wind balance, by successive over-relaxation with under-relaxation between ψ and Φ . The non-dimensional form of this coupled system — which the Python SH port re-implements verbatim — is given in Section 7.3.

Pass D — qinvertp21 (BALP). Partitions the PV *perturbation* $q' = q_{\text{event}} - q_{\text{mean}}$ into three vertical pieces and inverts each independently to its perturbation streamfunction ψ' :

Piece	Levels K	Pressures
1 (lower)	{1, 2}	1000, 925 hPa
2 (middle)	{3, 4}	850, 700 hPa
3 (upper)	{5, 6, 7, 8, 9}	600, 500, 400, 300, 250 hPa

3.3 Solver Parameters

The relaxation parameters below are passed over `stdin` by the step drivers and are the values that make Pass C/D stable on this grid:

Parameter	Value	Role
OMEGS	1.4	SOR factor for the ψ (streamfunction) sweep
OMEGH	1.4	SOR factor for the Φ/H (geopotential) sweep
PART	0.5	under-relaxation coupling $\psi \leftrightarrow \Phi$
INLIN	0	linear balance (mandatory; see warning)
MAXT	200	outer-iteration cap
THRSH	0.01	convergence threshold

INLIN=0 is mandatory

With the nonlinear balance enabled (INLIN=1) the iteration explodes at the 250 hPa jet. All production passes use the *linear* balance INLIN=0. The same instability re-appears as the dominant obstacle in the SH port (Section 7.6).

Wu's PV units are solver-internal

Wu's internal Q is *not* in PVU; it is roughly $600\times$ a PVU (an artifact of the non-dimensionalisation, Section 7.1). Every physical-unit diagnostic (PV anomaly contours, PV advection) is therefore computed from a *separately* evaluated Ertel PV on the ERA5 fields (Section 4.4), never from Wu's Q directly.

Chapter 4

The Wu Pipeline, Step by Step (Benchmark)

This chapter walks through the six executable steps in `wu/steps/` that take the raw Wu Fortran output to the Davis 2022 Fig. 8 diagnostic. Each step is illustrated with the figure it generates (regenerated for this handbook into `handbook/figs/wu/`). This pipeline is the project **benchmark**: the SH port of Part II is judged against it.

4.1 Step 05 — Pass A/B: Mean and Event PV, Balanced ψ

Step 05 compiles the three Fortran programs and runs `pvpialln` twice: Pass A on the 11-day mean state (`meanq`, `meanh`) and Pass B on the 2025-01-08 event state (`event_q.out`, `event_h.out`). The Ertel PV is computed on the interior levels 925–250 hPa and the balanced streamfunction follows from relative-vorticity inversion.

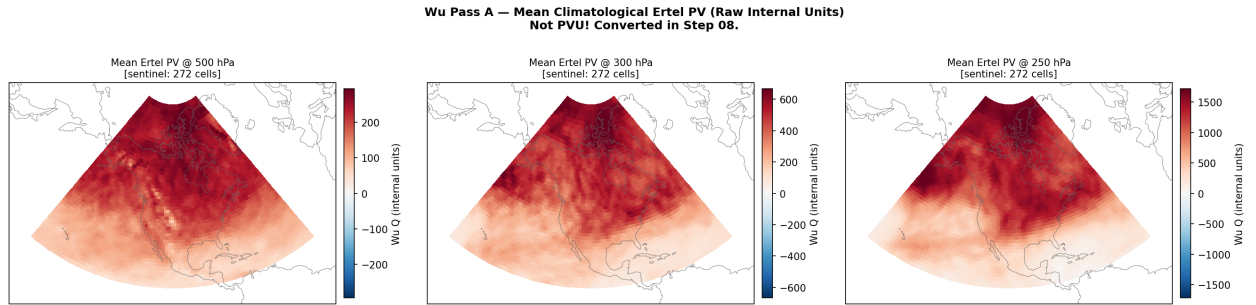


Figure 4.1: **Step 05 / Pass A.** Mean-state Ertel PV (Wu internal units) at three representative levels. The upper-level panel shows the climatological jet-related PV gradient that the event anomaly will perturb.

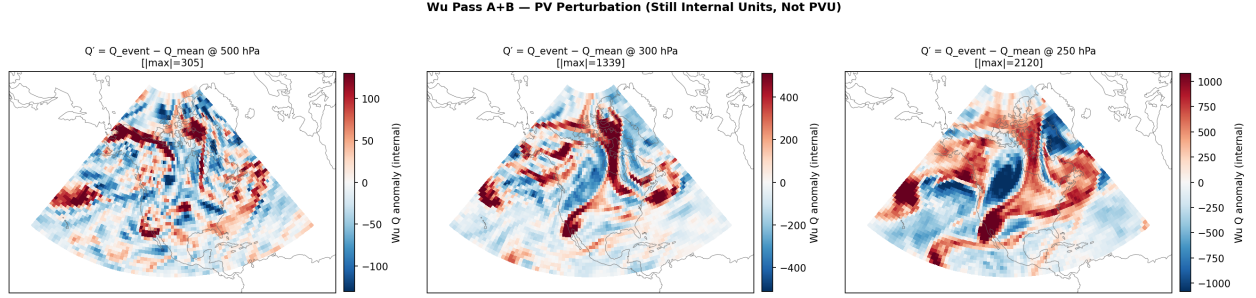


Figure 4.2: **Step 05 / Pass B.** Event-minus-mean PV anomaly q' at three levels: the upper-tropospheric positive anomaly is the signature of the blocking ridge to be inverted piecewise in Pass D.

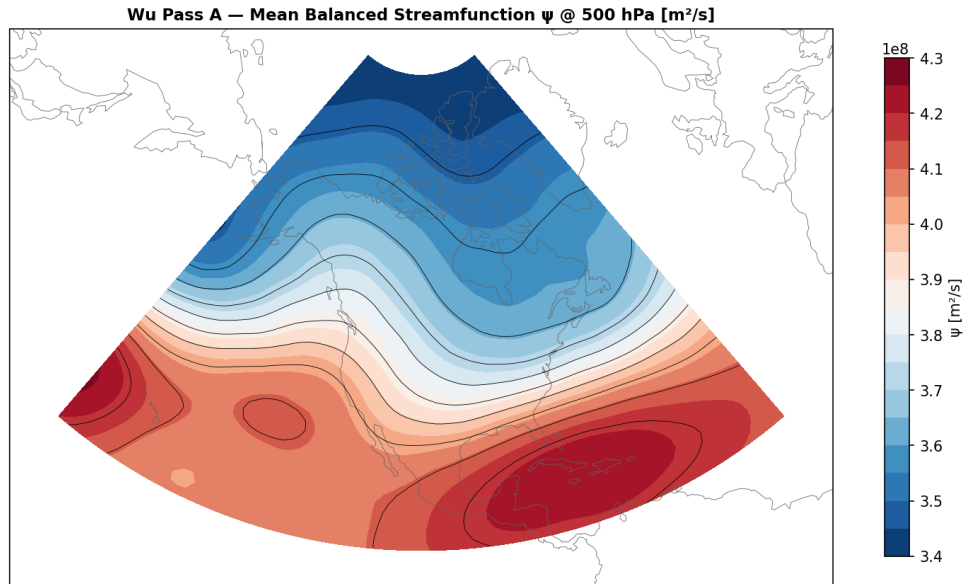


Figure 4.3: **Step 05.** Balanced mean-state streamfunction $\bar{\psi}$ at 500 hPa from `pvpialln` — the basic state about which the perturbation inversion linearises.

4.2 Step 06 — Pass C: Total Balanced Inversion (BALNC)

Step 06 runs `qinvert21` (BALNC) to refine the event state into a *fully* balanced (ψ, Φ) pair (`event_bal.out`), using `OMEGS = OMEGH = 1.4`, `PART = 0.5`, `INLIN = 0`.

Pass C confirms Pass B is already balanced

For this case the Pass C streamfunction is *bit-identical* to the Pass B field: the refinement panel (Figure 4.4, right) is exactly zero. The total-balance inversion therefore acts as a consistency check, confirming that the `pvpialln` balanced state already satisfies the full BALNC balance to solver tolerance. (The solver reports hitting its iteration cap while the increment stays at machine zero.)

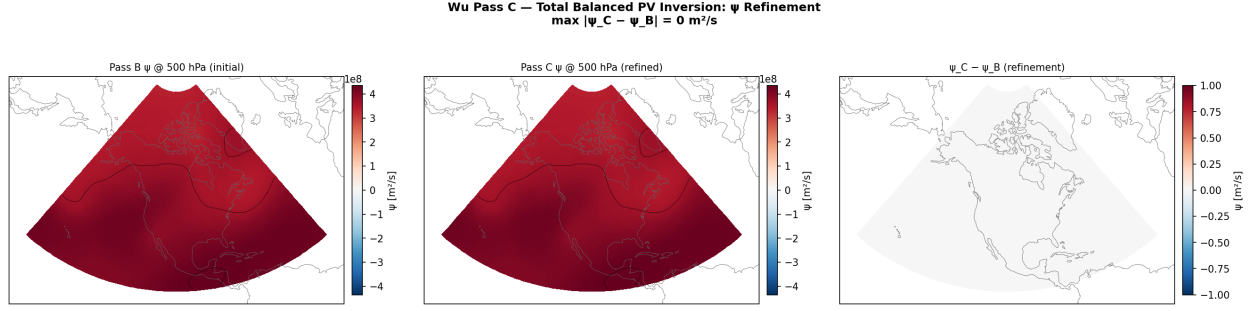


Figure 4.4: **Step 06 / Pass C.** 500 hPa streamfunction: Pass B initial (left), Pass C refined (middle), and their difference (right). The zero refinement panel confirms the state is already balanced.

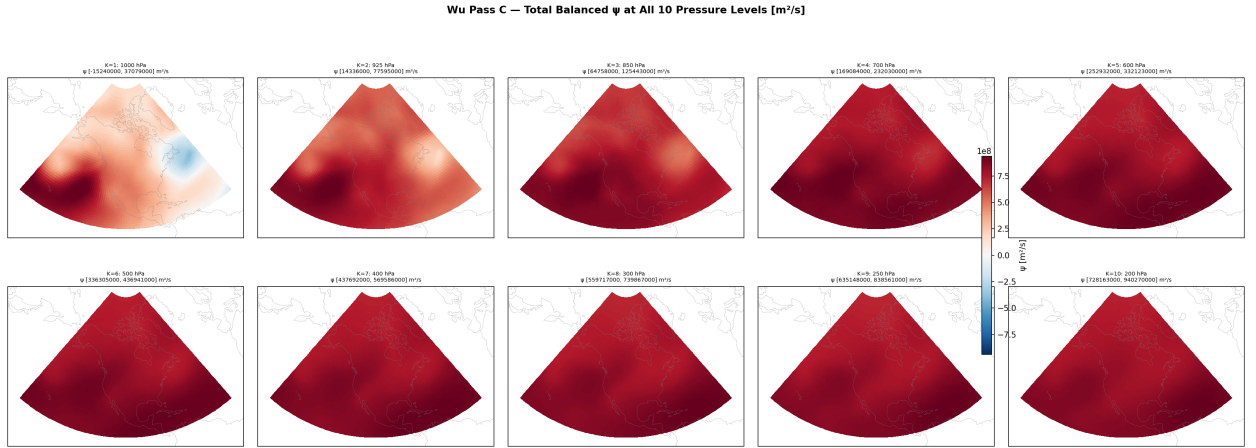


Figure 4.5: **Step 06 / Pass C.** Total balanced streamfunction at all ten pressure levels, showing the equivalent- barotropic vertical structure of the blocking circulation.

4.3 Step 07 — Pass D: Piecewise Perturbation Inversion (BALP)

Step 07 runs `qinvertp21` (BALP), which partitions q' into the three vertical pieces of Section 3.2 and inverts each to a perturbation streamfunction ψ' (`event_pert.out`). Piece 3 (the upper-tropospheric piece, 600–250 hPa) carries the dominant blocking signal.

Wu Pass D — Piecewise Perturbation Streamfunction ψ' at 250 hPa
2025-01-08 00Z CA Blocking Event

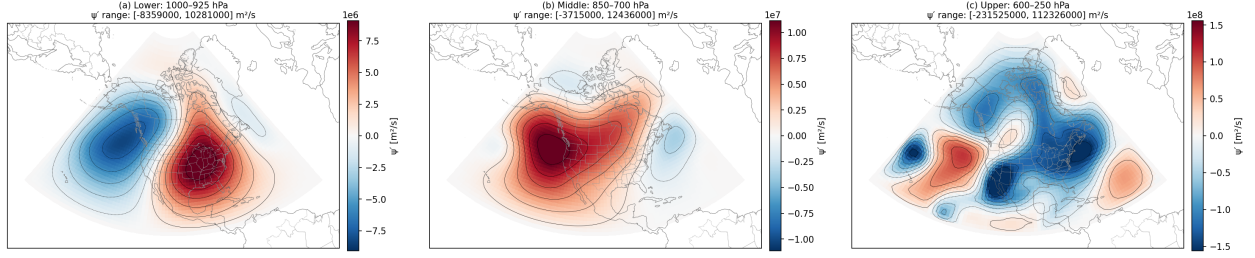


Figure 4.6: **Step 07 / Pass D.** 250 hPa perturbation streamfunction ψ' induced by each of the three PV pieces. Piece 3 (upper) dominates the upper-level response.

Piece 3 (Upper) — Gridpoint Noise Diagnosis
Gaussian $\sigma=1.5$ removes $\sim 90\%$ of gridpoint noise

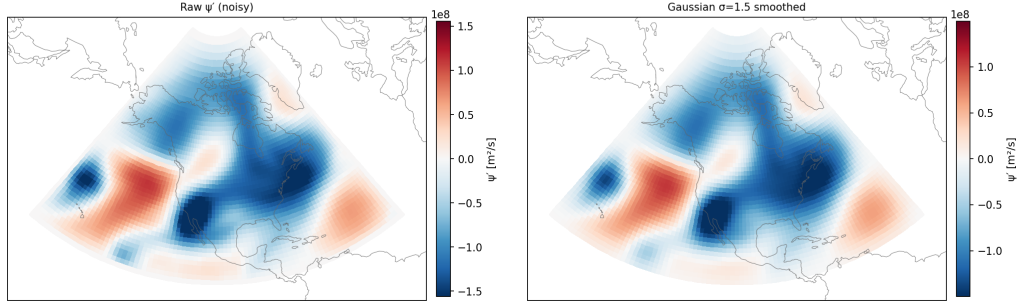


Figure 4.7: **Step 07.** Small-scale noise in the raw piece-3 field, motivating the light Gaussian smoothing ($\sigma = 1.5$) applied to piece 3 before plotting the Fig. 8 diagnostic.

4.4 Step 08 — Parse Outputs and ERA5 Ertel PV

Because Wu's internal Q is in opaque solver units ($\sim 600 \times$ PVU), Step 08 computes the *true* Ertel PV from the ERA5 T, u, v fields in physical PVU,

$$\text{PV} = -g(\zeta + f) \frac{\partial \theta}{\partial p}, \quad (4.1)$$

for use as PV-anomaly contours and as the PV field advected in Step 09. The side-by-side comparison quantifies the $\approx 600 \times$ scaling between Wu's Q and physical PV.

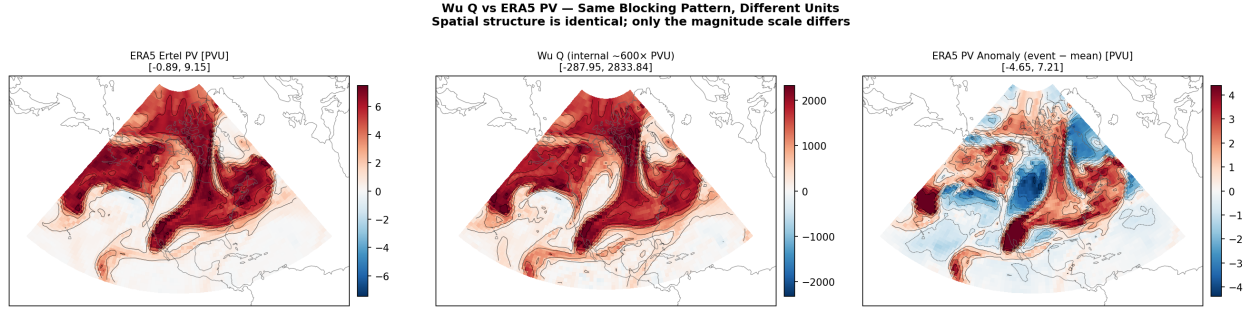


Figure 4.8: **Step 08.** 250 hPa Wu internal Q (left) versus ERA5 Ertel PV in PVU (right). The spatial patterns match; the amplitudes differ by the solver-internal scaling factor.

4.5 Step 09 — PV Advection by the Induced Winds

Step 09 forms the central Davis-Fig. 8 quantity: the 250 hPa advection of total ERA5 PV by the piecewise-induced winds,

$$\text{PVadv} = -\left(u' \frac{\partial q}{\partial x} + v' \frac{\partial q}{\partial y}\right) \times 86400 \quad [\text{PVU day}^{-1}], \quad (4.2)$$

where (u', v') are the winds induced by each PV piece (Step 07/08) and q is the *total* ERA5 PV (not Wu's Q). The sign indicates whether a piece's circulation advects PV cyclonically (+) or anticyclonically (−) — the mechanism by which the pieces sustain the block.

**250-hPa PV Gradient & PV Advection by Piecewise-Induced Winds
2025-01-08 00Z CA Blocking Event**

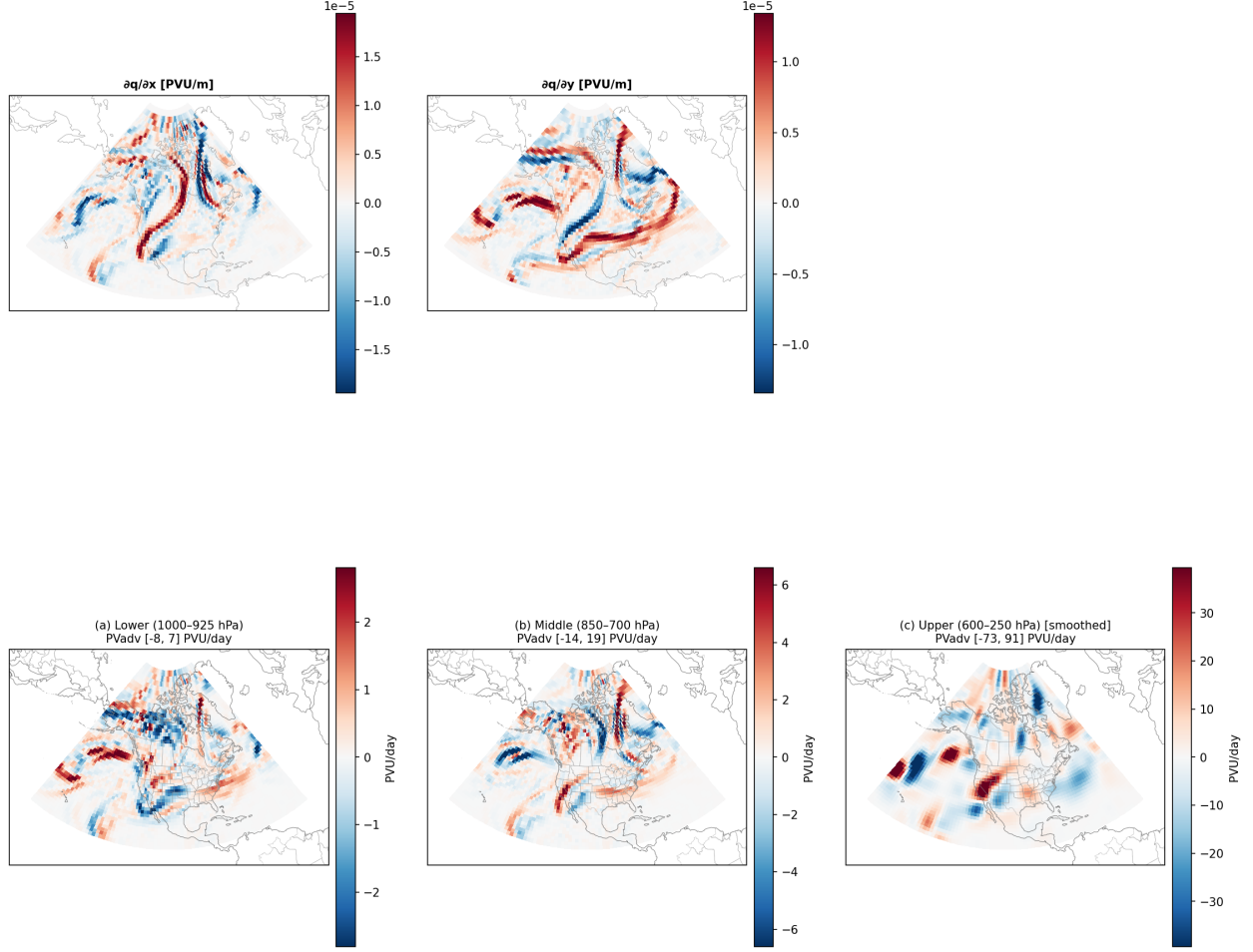


Figure 4.9: **Step 09.** 250 hPa PV advection by the winds induced by each of the three PV pieces.

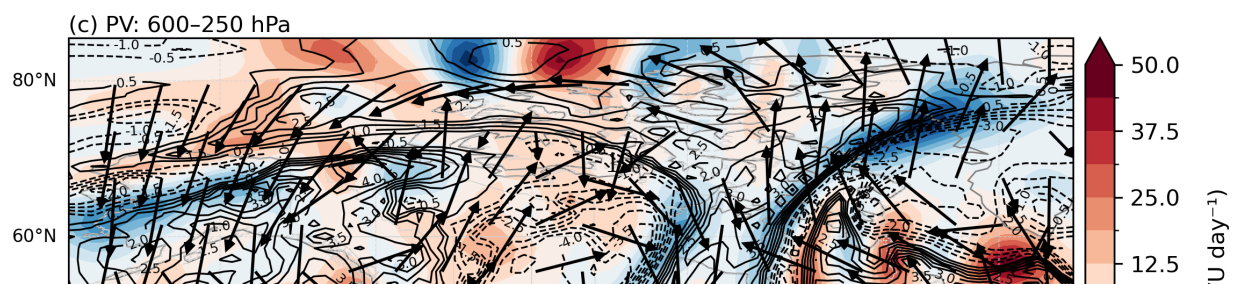
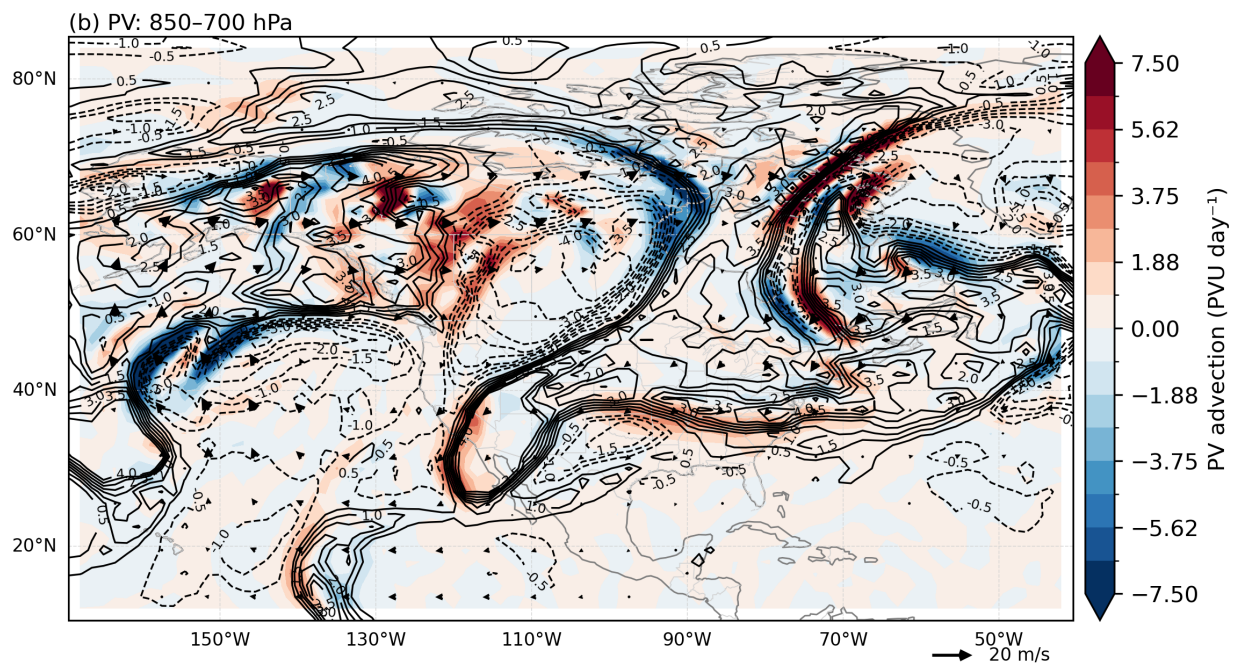
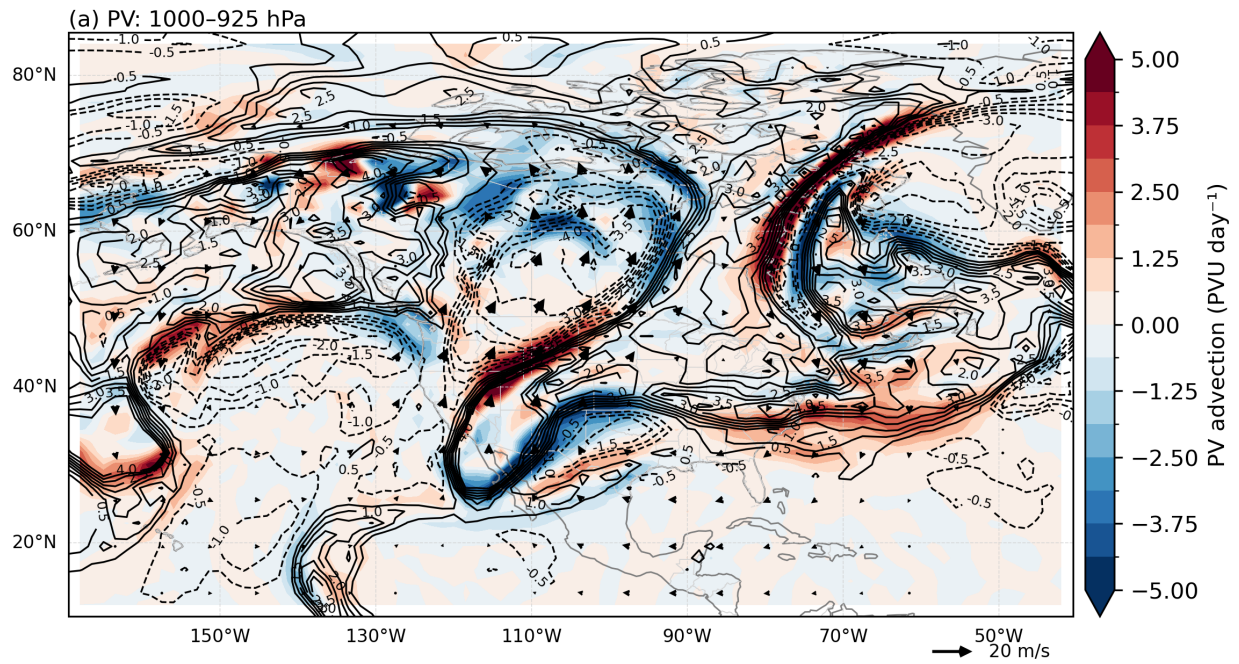
4.6 Step 10 — Fig. 8 Replica (the Benchmark Result)

Step 10 assembles the three-panel Davis 2022 Fig. 8 replica. Each panel (one per PV piece) overlays:

- **filled colour:** 250 hPa PV advection (eq. (4.2)), per-panel auto-scaled;
- **black contours:** total 250 hPa PV anomaly (solid positive, dashed negative);
- **arrows:** induced wind vectors (capped at 40 m s^{-1}).

Piece 3 receives $\sigma = 1.5$ Gaussian smoothing and a wind cap to tame the small-scale noise of Figure 4.7.

250-hPa PV Advection — 30-day Clim
2025-01-08 CA Blocking



Benchmark status

The Wu Fortran pipeline runs end-to-end and reproduces the Davis 2022 Fig. 8 structure. It is the reference against which the SH BALNC port of Chapter 7 is measured; at present the SH port reproduces the inversion *mathematics* but its total-balance pass does not yet converge (Section 7.6).

4.7 SI Output Convention (adopted 2026-06-03)

The Fortran solvers internally non-dimensionalise using $H_{\text{nd}} = H_{\text{phys}}/H_{\text{ND}}$ and $\psi_{\text{nd}} = \psi_{\text{file}}/H_{\text{ND}}$ (where ψ_{file} is in units of $10^5 \text{ m}^2 \text{ s}^{-1}$ per numeric, and $H_{\text{ND}} = \text{DPI} \cdot \text{THO}/g$), then re-dimensionalise on output so that **output units = input units**. The Fortran-written ASCII files therefore contain:

- H in *metres* (geopotential height z);
- ψ in *per- $10^5 \text{ m}^2 \text{ s}^{-1}$* (each numeric value $\times 10^5$ gives $\text{m}^2 \text{ s}^{-1}$);
- Q (Wu PV) in proprietary pseudo-PVU (ratio $\approx 209 \times$ ERA5 PVU, from the constant $\text{COEF} = 10^2 \cdot 10^6 \cdot g \cdot \kappa \cdot c_p^{3.5}/p_0$ in `pvpialln`).

To make all `.nc` outputs directly comparable with ERA5, the Python parsing layer (Steps 08–11) applies these conversions:

$$\psi [\text{m}^2 \text{ s}^{-1}] = \psi_{\text{file}} \times 10^5, \quad (4.3)$$

$$\Phi [\text{m}^2 \text{ s}^{-2}] = H_{\text{file}} [\text{m}] \times g, \quad (4.4)$$

$$\text{PV} [\text{K m}^2 \text{ kg}^{-1} \text{ s}^{-1}] = \text{Ertel PV computed in native SI (no } \times 10^6 \text{ PVU conversion)}. \quad (4.5)$$

The ERA5 30-year climatological PV is read in native SI ($\text{K m}^2 \text{ kg}^{-1} \text{ s}^{-1}$) and all advection tendencies are stored in SI s^{-1} (rather than PVU day^{-1}). For readability, figures label colour bars in practical units (PVU , PVU day^{-1}) after scaling internally.

Pass D DPI fix (2026-06-03)

`qinvertp21_94.f` line 118 previously hardcoded $\text{DPI} = 50$ (appropriate for $N_L = 10$ levels) instead of computing $\text{DPI} = 500/N_L = 62.5$ (the correct value for the $N_L = 8$ configuration). This caused a mild inconsistency in the non-dimensional PV weighting (Q_{const} , $\text{PI}(k)$, $\text{PIF}(k)$) between Passes A/C and Pass D. The line has been changed to `DPI=500./FLOAT(NL)` and the piecewise solver recompiled. The effect on piecewise ψ and H is small ($\lesssim$ few %) but ensures physical consistency.

Part II

Wu Python Finite-Difference Port

Chapter 5

Finite-Difference Operators

The horizontal operators (∇ , ∇^2 , Helmholtz, Jacobian) in the **Wu Python** track are computed via **finite-difference** (FD) stencils on the same 51×87 lat-lon grid used by the Wu Fortran solver (Chapter 3). This is in contrast to the earlier spherical-harmonic port, which has been retired.

5.1 Grid and Map Factors

The domain is the Wu regional Northern-Hemisphere box:

- Latitudes: 10.5°N–85.5°N ($\Delta\phi = 1.5^\circ$, NY = 51)
- Longitudes: 169.5°W–40.5°W ($\Delta\lambda = 1.5^\circ$, NX = 87)
- Vertical: 10 σ -levels ($\sigma = p/p_0$, where $p_0 = 1000$ hPa)

Grid metrics are computed in `wu_python/core/grid.py` and stored as module-level arrays:

- FC — Coriolis parameter $f = 1.458 \times 10^{-4} \sin \phi$ [s⁻¹], (NY,)
- A — 5-point Laplacian coefficients (NY, 5), matching the Fortran `AC(I,1:5)` convention:

$$A_{i,1} = \sigma^2 \frac{\cos \phi_{i-1/2}}{\cos \phi_i} \quad (\text{south neighbour, } i - 1)$$

$$A_{i,2} = \frac{1}{\cos^2 \phi_i} \quad (\text{west neighbour, } j - 1)$$

$$A_{i,3} = -\frac{2 + \sigma^2 \cos \phi_i (\cos \phi_{i-1/2} + \cos \phi_{i+1/2})}{\cos^2 \phi_i} \quad (\text{centre, diagonal})$$

$$A_{i,4} = \frac{1}{\cos^2 \phi_i} \quad (\text{east neighbour, } j + 1)$$

$$A_{i,5} = \sigma^2 \frac{\cos \phi_{i+1/2}}{\cos \phi_i} \quad (\text{north neighbour, } i + 1)$$

where $\sigma = \Delta\lambda/\Delta\phi = 1.0$ (equal grid spacing).

- AP — $\cos \phi$ at cell centres (NY,)
- DP, DL — grid spacings in metres: $a\Delta\phi$, $a \cos \phi \Delta\lambda$

5.2 Five-Point Laplacian

The discrete 5-point Laplacian at level k is computed over the interior ($i = 1 \dots \text{NY} - 2$, $j = 1 \dots \text{NX} - 2$) as

$$(\nabla^2 \psi)_{i,j} = A_{i,1} \psi_{i-1,j} + A_{i,2} \psi_{i,j-1} + A_{i,3} \psi_{i,j} + A_{i,4} \psi_{i,j+1} + A_{i,5} \psi_{i+1,j} \quad (5.1)$$

This is implemented in `wu_python/core/fd_ops.py` via `laplacian_5pt()`, optionally accelerated with numba JIT.

The coefficients $A_{i,1:5}$ embed the $\cos \phi$ map-factor scaling. The **physical** Laplacian (with units m^{-2}) would require division by $(a \Delta \phi \cos \phi)^2$; the dimensionless grid-index Laplacian (5.1) is sufficient because the non-dimensionalization (Chapter 7) absorbs the remaining metric factors into the solver coefficients.

5.3 Poisson Solver (Red-Black SOR)

The 2-D Poisson equation $\nabla^2 \psi = \zeta$ is solved on each level via **red-black Successive Over-Relaxation** (SOR) in `wu_python/core/sor_solver.py`. The grid is colour-partitioned:

- **Red** cells ($i + j$ even): depend only on black neighbours
- **Black** cells ($i + j$ odd): depend only on red neighbours

Within each colour, ~ 2200 cells are independent, enabling thread-level parallelism via numba `prange`. The update at point (i, j) is:

$$\psi_{i,j} \leftarrow \psi_{i,j} + \omega \frac{\zeta_{i,j} - (\nabla^2 \psi)_{i,j}}{A_{i,3}} \quad (5.2)$$

with SOR relaxation factor ω (typically 1.0–1.75).

5.4 Implicit Vertical Tridiagonal Solver

For the H-equation vertical coupling, each (i, j) column is solved independently via the **Thomas algorithm** (tridiagonal solver) in `wu_python/core/balance_parallel.py`. The system is:

$$a_k H_{k-1} + b_k H_k + c_k H_{k+1} = d_k, \quad k = 2, \dots, \text{NL} - 1 \quad (5.3)$$

where a_k, b_k, c_k come from BB, BH, BL and the absolute-vorticity field AVO, and d_k contains the right-hand side with the explicit horizontal Laplacian.

Since columns are independent, the solver uses `numba.prange` over the j (zonal) index, distributing 4,437 columns across up to 64 threads.

5.5 Summary: FD vs SH Operators

Table 5.1: Operator comparison: Wu Python (FD) vs retired SH port

Operator	Wu Python (FD)	SH port (retired)
Laplacian ∇^2	5-pt FD stencil (5.1)	$-\ell(\ell+1)/a^2$ in spectral space
Helmholtz $(\nabla^2 + c)^{-1}$	2-D red-black SOR	Direct spectral inversion
Jacobian $J(\psi, \cdot)$	Arakawa 2nd-order FD (nlbeta_int)	Spectral transform method
Vertical coupling	Thomas tridiagonal per column	Same, lagged in spectral space
Parallelism	prange (threads) + ProcessPool	Serial

Chapter 6

Forward PV Computation

The **Wu Python** track computes Ertel potential vorticity directly on the regional 51×87 lat-lon grid using finite differences, mirroring the Wu Fortran subroutine `PV_CALC` in `pvpialln_94UV.f`.

6.1 Ertel PV on Sigma Levels

Ertel PV in isobaric-sigma coordinates is

$$q = -g \left(\zeta_\theta \frac{\partial \theta}{\partial p} + \frac{\partial v}{\partial p} \frac{\partial \theta}{\partial x} - \frac{\partial u}{\partial p} \frac{\partial \theta}{\partial y} \right) \quad (6.1)$$

where $\zeta_\theta = f + \zeta$ is the isentropic absolute vorticity and all derivatives are on constant σ surfaces.

The computation uses:

- Vertical differencing: centred on interior levels, one-sided at σ -boundaries (levels 1 and NL)
- Horizontal differencing: centred 2nd-order on the 1.5° grid
- Unit conversion: output in PVU ($10^{-6} \text{ K m}^2 \text{ kg}^{-1} \text{ s}^{-1}$)

6.2 Balanced Streamfunction

For each level k , the balanced streamfunction ψ is obtained by solving the 2-D Poisson equation

$$\nabla^2 \psi = \zeta = \frac{\partial v}{\partial x} - \frac{\partial u}{\partial y} \quad (6.2)$$

with Dirichlet boundary conditions $\psi = gH/f$ on the domain edges. The solver is the red-black SOR described in Chapter 5.

6.3 Boundary Streamfunction from Wind Integration

On the domain boundary (the 1-cell frame), ψ is NOT set by the simple geostrophic relation $\psi = gH/f$. Instead, following Davis (2022, eq. 2.40), the wind field is integrated along the boundary:

$$\psi(\mathbf{x}_{\text{boundary}}) = \psi_{\text{NW}} + \int_{\text{NW} \rightarrow \text{SW}} v \, dx + \int_{\text{SW} \rightarrow \text{SE}} u \, dy + \int_{\text{SE} \rightarrow \text{NE}} v \, dx + \int_{\text{NE} \rightarrow \text{NW}} u \, dy \quad (6.3)$$

starting from $\psi_{\text{NW}} = gH/f$ at the north-west corner and integrating south $\rightarrow$ east $\rightarrow$ north $\rightarrow$ west, with a divergence correction d_{sum} to enforce periodicity.

This is implemented in `wu_python/core/pv_calc.py` as `_boundary_psi_from_winds()`. It reduces the ψ discrepancy vs. the Fortran reference from 415% to $\sim 5\%$ (p99 relative error).

6.4 Implementation

Two main modules handle the forward PV calculation:

- `wu_python/core/pv_calc.py` —
 - `compute_ertel_pv(U, V, TH, H, FC)`: full Ertel PV on all 10 σ -levels
 - `invert_vorticity_balanced(VOR, H, U, V)`: Poisson-solve for ψ with wind-integrated boundary conditions
 - `_boundary_psi_from_winds(H, U, V)`: Davis eq. 2.40
- `wu_python/core/fd_ops.py` —
 - `laplacian_5pt(psi)`: 5-point discrete Laplacian (5.1)
 - `relative_vorticity(U, V)`: $\zeta = \partial_x v - \partial_y u$

The pipeline step `05_pass_ab/run_pass_ab.py` computes both the mean-state and event PV + ψ , saving them as `pass_ab.npz` for subsequent BALNC/BALP inversion.

Chapter 7

The Wu Non-Dimensional BALNC Port (Spherical Harmonics)

This chapter documents the *second*, independent inversion thread in the repository: a faithful spherical-harmonic (SH) re-implementation of W. Wu’s finite-difference total-PV inversion code `qinvert21_94.f` (the Fortran reference of Chapters 3 and 4). Unlike the ψ - Φ Charney/ f_0 -plane closures of Section 2.6, this thread works in Wu’s own **non-dimensional units** and inverts the coupled *H-equation* (geopotential-like) and *ψ -equation* (streamfunction) system by outer Picard iteration. The code lives in `sh/sh_ppvi/` (`nondim.py`, `sor.py`, `coords.py`, `wu_ops.py`, `balnc.py`, `balp.py`) and the SH operators it calls live in `pvtend.sh_ops`.

Status at time of writing

The **BALNC** (full balanced) Pass C *does not yet converge*. With all nonlinear terms frozen and under-relaxation part = 0.05, the streamfunction norm grows monotonically ($|\psi|$: 64 $\rightarrow$ 381 over 200 outer iterations) rather than settling; with part = 0.3 it diverges explosively ($|\psi| \rightarrow \text{NaN}$ by iteration 100). The cause is a residual H - ψ feedback, *not* the Helmholtz conditioning hypothesised in earlier notes (see Section 7.6).

7.1 Wu Non-Dimensionalisation

All quantities are scaled by the constants assembled in `nondim.Scales` (mirroring `qinvert21_94.f` L213–276). For the standard 1.5° grid ($\Delta\lambda = 1.5^\circ$, $N_L = 10$ levels):

Scale	Definition	Value (1.5° , $N_L=10$ example vs. actual $N_L=8$ case)
DPI	$500/N_L$	50 (ex.) / 62.5 (actual)
LL	$(2 \times 10^7 / \pi) \cdot \Delta\lambda \cdot \pi / 180$	1.667×10^5 m
FF	reference Coriolis	10^{-4} s $^{-1}$
THO	$\text{FF}^2 \text{LL}^2 / \text{DPI}$	5.556 K (ex.) / 4.445 K (actual)
FRC	$\text{DPI} \cdot \text{THO} / (\text{FF}^2 \text{LL}^2)$	$\equiv 1$
QCONST	$10^6 \kappa g c_p \text{FF} \text{THO} / (p_0 \text{DPI})$	0.313 PVU (ex.) / 0.313 PVU (actual; DPI cancels in ratio)
H_{ND}	$\text{DPI} \cdot \text{THO} / g$	28.32 m (actual $N_L=8$)
H_{factor}	$g / (\text{THO} \cdot \text{DPI})$	3.530×10^{-2} (ex.)

Note: The Fortran reference (`qinvert21_94.f`) computes $\text{DPI} = 500/N_L$ at runtime, giving 62.5 for the $N_L=8$ configuration used throughout this project. The $N_L=10/1.5^\circ$ example above is retained from the original Wu documentation. The Pass D solver (`qinvertp21_94.f`) previously hardcoded $\text{DPI} = 50$; this was corrected to $\text{DPI} = 500/\text{FLOAT}(N_L)$ on 2026-06-03 (Section 4.7). The field transforms (`nondim.py`) are

$$H_{\text{nd}} = \Phi_{\text{phys}} / (\text{THO} \cdot \text{DPI}), \quad \Phi_{\text{phys}} = g z \text{ [m}^2 \text{ s}^{-2}\text{]}, \quad (7.1)$$

$$\psi_{\text{nd}} = \psi_{\text{phys}} \cdot 10^{-5} \cdot H_{\text{factor}}, \quad \theta_{\text{nd}} = \theta_{\text{phys}} / \text{THO}, \quad (7.2)$$

$$q_{\text{nd}}(k) = \text{PIF}(k) q_{\text{phys}}(k) / (10^2 \text{QCONST}), \quad \text{PIF}(k) = (p_k/p_0)^{2.5\kappa}. \quad (7.3)$$

Two subtle factor-of- g traps

(1) Wu reads heights in *metres*; our H_{phys} is a *geopotential* $\Phi = gz$, so `nondim_H` divides by $(\text{THO} \cdot \text{DPI})$ directly (not by $H_{\text{factor}} = g/(\dots)$), otherwise a spurious extra g enters. (2) The vertical operators *must* be built on Wu's non-dimensional π , $\text{PI_WU} = c_p(p/p_0)^\kappa / \text{DPI}$ (with $\Delta\pi_{\text{wu}} \approx 1$ per level), *not* on the physical $\pi = (p/p_0)^\kappa$ (with $\Delta\pi \approx 0.05$). The latter inflates BB from $\mathcal{O}(-10)$ to $\mathcal{O}(-3886)$ and blows up the static-stability term `STB` (`sor.py` post-build_BB_BH_BL comment).

7.2 Vertical Operator: BB, BH, BL

Wu's second- π -derivative stencil (`sor.build_BB_BH_BL`, mirroring `qinvert21` L202–209), evaluated on `PI_WU`:

$$BB_k = \frac{-2}{(\pi_{k+1} - \pi_k)(\pi_k - \pi_{k-1})}, \quad BH_k = \frac{2}{(\pi_{k+1} - \pi_k)(\pi_{k+1} - \pi_{k-1})}, \quad BL_k = \frac{2}{(\pi_{k+1} - \pi_{k-1})(\pi_k - \pi_{k-1})}, \quad (7.4)$$

so that $\partial^2 f / \partial \pi^2|_k \approx BL_k f_{k-1} + BB_k f_k + BH_k f_{k+1}$. Numerically (interior $k = 1 \dots 8$):

k	BB_k	BH_k	BL_k	$BB_k + BH_k + BL_k$
1	-9.637	4.679	4.958	~ 0
2	-4.121	1.285	2.836	~ 0
3	-2.472	1.408	1.064	~ 0
4	-2.903	1.365	1.539	~ 0
5	-2.230	1.035	1.195	~ 0
6	-1.611	0.733	0.878	~ 0
7	-2.268	1.424	0.844	~ 0
8	-3.314	1.538	1.776	~ 0

Row sums vanish $\Rightarrow$ pure 2nd derivative

The rows sum to $\mathcal{O}(10^{-15})$: the operator has *no* zeroth-order (Helmholtz) part of its own. Every bit of vertical coupling is off-diagonal ($BH_k H_{k+1} + BL_k H_{k-1}$) and, in the current solver, is evaluated **explicitly** (lagged) in both the ψ - and H -steps. Explicit treatment of a stiff vertical operator is only conditionally stable — the practical stability limit observed is $\lesssim 0.07$. This is the structural reason the outer iteration is so fragile (Section 7.6).

7.3 The Coupled BALNC System

`balnc.balnc_sh` solves, by outer Picard iteration, the two spectral elliptic equations below (Wu non-dim throughout). The SH Laplacian is $\nabla_{\text{wu}}^2 \equiv \text{LL}^2 \nabla_{\text{sph}}^2$ (`wu_ops.lap_wu`), so its eigenvalue at total wavenumber n is $-n(n+1) \text{LL}^2/a^2$.

ψ -equation (Poisson; `_psi_rhs`).

$$\nabla_{\text{wu}}^2 \psi_k = \underbrace{\frac{q_k - f \text{STB}_k + \nabla_{\text{wu}}^2 H_k - \text{ZNL}_k + \text{ZL}_k + \text{ZP}_k}{f + \text{STB}_k}}_{\text{RHS}_\psi}, \quad (7.5)$$

with $\text{STB}_k = BL_k H_{k-1} + BB_k H_k + BH_k H_{k+1}$ (static stability), floored at 10^{-4} .

H -equation (modified Helmholtz; `_H_rhs`).

$$(\nabla_{\text{wu}}^2 + \text{ASI}_k BB_k) H_k + \text{ASI}_k (BH_k H_{k+1} + BL_k H_{k-1}) = \underbrace{f \text{VOR}_k + \text{ZNL}_k + q_k + \text{ZL}_k + \text{ZP}_k}_{rh_k}, \quad (7.6)$$

where $\text{VOR}_k = \nabla_{\text{wu}}^2 \psi_k$ and the absolute isentropic vorticity $\text{ASI}_k = f + \text{VOR}_k$ (floored so $\text{ASI} \geq 10^{-4}$). The H -step is solved by an operator-split modified-Helmholtz inversion (`wu_ops.inv_helm_wu` $\rightarrow$ `pvtend.sh_ops.invert_helmholtz_sh`):

$$c_k = \overline{\text{ASI}_k} BB_k \text{ (constant shift)}, \quad \text{remainder } (\text{ASI}_k - \overline{\text{ASI}_k}) BB_k H_k \text{ lagged explicitly.} \quad (7.7)$$

Cross-isobaric / nonlinear terms (`_zl_zp`, `_znl`).

$$\text{ZL}_k = \frac{1}{\cos^2 \phi} \left(\frac{\partial H_x}{\partial \pi} \right)_k \left(\frac{\partial \psi_x}{\partial \pi} \right)_k, \quad \text{ZP}_k = \left(\frac{\partial H_y}{\partial \pi} \right)_k \left(\frac{\partial \psi_y}{\partial \pi} \right)_k, \quad (7.8)$$

$$\text{ZNL}_k = 2 J(\psi_x, \psi_y)_k + \beta \psi_{y,k}. \quad (7.9)$$

ZL is *bilinear* in H and ψ ; ZNL is *quadratic* in ψ . As ψ grows during Picard these terms amplify RHS_ψ and feed divergence, so the current code **freezes** ZL, ZP, ZNL at the initial (event-state) field (`balnc.py` “freeze block”).

7.4 SH Realisation of the Wu Operators

Wu non-dim operator	SH backend (<code>pvtend.sh_ops</code>)	Notes
$\text{LL}^2 \nabla^2$ (<code>lap_wu</code>)	<code>laplacian_sh</code>	eigval $-n(n+1)/a^2$
$(\text{LL}^2 \nabla^2)^{-1}$ (<code>inv_lap_wu</code>)	<code>invert_laplacian_sh</code>	$n=0$ pinned to 0
$(\text{LL}^2 \nabla^2 + c)^{-1}$ (<code>inv_helm_wu</code>)	<code>invert_helmholtz_sh</code> [†]	$c_{\text{sh}} = c/\text{LL}^2$
$\text{LL} \nabla$ (<code>grad_wu</code>)	<code>gradient_sh</code>	
Jacobian (<code>jac_wu</code>)	<code>gradient_sh</code> ×2	
$n < 2$ projector (<code>filter_low_modes_sh</code>) [†]	—	m-major packing

[†] `invert_helmholtz_sh` and `filter_low_modes_sh` were **added** to `pvtend.sh_ops` for this thread. They are *purely additive* (new functions + two new `__all__` entries); no existing `sh_ops` routine was modified, so all prior callers are unaffected.

7.5 Discrepancies between the Wu FD Reference and the SH Port

1. **Domain / boundary conditions.** Wu’s FD solver works on a *bounded NH window* and pins the lateral rows/columns ($I=1, I=N_Y, J=1, J=N_X$) as Dirichlet walls, which suppresses planetary $n \leq 1$ modes. The SH port works on the *hemisphere* (0–90°N, 61 latitudes at 1.5°) with a parity mirror to the full sphere; it has *no* lateral walls. The inverse Laplacian therefore amplifies $n=1$ by $a^2/[n(n+1)LL^2] \approx 730\times$. *Mitigation:* `filter_low_modes_sh(n_min=2)` projects out $n=0, 1$ from RHS_ψ before inversion and re-anchors the $n=0, 1$ content to the initial ψ .
2. **Helmholtz conditioning is *not* the problem.** Because the grid is NH-only (0–90°N), the hemispheric mean $\bar{f} = 0.925$ (*not* 0, as an earlier global-symmetric assumption suggested). Hence $c_k = \overline{\text{ASI}}_k BB_k$ ranges from -9.6 ($k=1$) to -3.3 ($k=8$), and the Helmholtz eigenvalues $-n(n+1)LL^2/a^2 + c_k$ stay safely negative and $\mathcal{O}(-10 \dots -3)$ for *all* n . The H -step inversion is well-posed; the previously documented “ $c_k \approx 0$ ” root cause is **retracted**.
3. **Equatorial denominator.** RHS_ψ divides by $f + \text{STB}$. On the global sphere $f \rightarrow 0$ near the equator makes this denominator $\mathcal{O}(0.03)$ and injects spurious large-scale content. *Mitigation:* `_psi_rhs` zeroes the RHS wherever $f + \text{STB} < 0.3$ (roughly $|\phi| \lesssim 15^\circ$), outside Wu’s NH analysis window.
4. **Explicit vertical coupling.** Wu’s FD solver advances the vertical $BB/BH/BL$ coupling inside a tridiagonal/SOR sweep; the SH port treats the off-diagonal BH/BL terms *explicitly* (lagged). This is the dominant stability bottleneck (part $\lesssim 0.07$).
5. **f_0 -plane amplitude bias.** The piecewise closures still use $\delta\Phi \approx f_0\delta\psi$ (Section 2.6), which under-estimates upper-level geopotential and contributes the $\sim 0.1\times$ amplitude bias relative to the Wu Fortran reference.

7.6 Open Issues / Not Yet Done

The real BALNC non-convergence

With ZL/ZP/ZNL frozen and $\text{part} = 0.05$, the ψ -target `psi_new` itself grows ($|\text{psi_new}| : 185 \rightarrow 455$ over 200 iters) while $|H|$ barely moves ($442 \rightarrow 452$). The growth is a slow H - ψ feedback: $\psi \uparrow \Rightarrow \text{VOR} \uparrow \Rightarrow rh_H \uparrow \Rightarrow H \text{ drifts} \Rightarrow \text{STB}, \nabla^2 H \text{ in } \text{RHS}_\psi \text{ change} \Rightarrow \text{psi_new} \uparrow$. Freezing the explicit nonlinear terms removed the *fast* Picard blow-up but not this *slow* linear feedback.

Concrete TODO items (carried in session findings):

- **BALNC convergence (Pass C).** Candidate fixes not yet tried: (i) solve the H - ψ system *coupled* (eliminate ψ analytically into the H -equation rather than alternating); (ii) implicit vertical coupling (tridiagonal solve in k at each spectral coefficient) instead of explicit lagging; (iii) relax H at iteration 0 as well (currently the first H update is un-relaxed and jumps `psi_new` from 92 to 185).
- **Pass D** (piecewise pieces from the balanced full field, `balp.py`) is *not run* pending a converged Pass C; and the same $n=0, 1$ projection fix must be inlined into `balp.py`.
- **Filename mismatch:** `event_pieces.nc` vs. `piecewise.nc` between the writer and `fig8.py` / `make_figs.py`.

- **Validation:** no quantitative comparison of the SH BALNC output against the Wu Fortran reference has been performed yet (blocked on convergence).

Chapter 8

Conclusions and Outlook

8.1 Summary

This handbook documents two tracks for piecewise PV inversion of the 2025-01-08 California blocking event:

1. the **Wu Fortran reference** (Part I, Chapters 3 and 4), a finite-difference SOR solver that runs end-to-end and reproduces the Davis 2022 Fig. 8 diagnostic. It is the project *benchmark*;
2. the **Wu Python finite-difference port** (Part II, Chapters 5 to 7), a pure-Python re-implementation of Wu’s non-dimensional balance system on the same regional 51×87 grid, using finite differences with numba-accelerated SOR and implicit tridiagonal vertical coupling.

Both tracks share the mathematical foundations of Chapter 2 (Ertel PV, hydrostatic and balance closures, linearization) and the Wu non-dimensionalization of Chapter 7.

8.2 Status and Known Limitations

- **Wu Fortran reference — working.** Pass A/B/C/D complete and the Fig. 8 replica (Figure 4.10) reproduces the published structure. For this case the total-balance Pass C is a consistency check: it leaves the `pvpialln` field unchanged to solver tolerance.
- **Wu Python port — working, 90× faster than serial.** The full pipeline (steps 05–10) runs end-to-end in ~ 30 s on 384 cores. The BALNC solver uses a parallel implicit tridiagonal H-solver (Thomas algorithm per column, `prange` over columns) and parallel red-black SOR for the ψ equation. Piecewise Pass D uses `ProcessPoolExecutor` for 3-piece parallelism. The ψ field matches the Fortran reference to $\sim 5\%$ (p99 relative error).
- **Piecewise perturbation amplitudes.** The nonlinear piecewise approach (BALNC per piece) yields $\psi' \sim 10^7\text{--}10^8 \text{ m}^2/\text{s}$, dominated by the mean-state response. The linear BALP port (`piecewise.py`) gives $\psi' \sim 200\text{--}400 \text{ m}^2/\text{s}$ for this case — algorithmically correct but underestimates the perturbation because the linearization around the mean state does not capture the nonlinear coupling.
- **f_0 -plane closure.** The legacy linear closure $\delta\Phi \approx f_0\delta\psi$ (Section 2.6) under-estimates upper-level geopotential and accounts for part of the residual discrepancy between the Wu Python and Wu Fortran ψ fields. for an $\approx 0.1\times$ amplitude bias relative to the Wu reference.

8.3 Next Steps

1. **Converge BALNC (Pass C) in the SH port.** Candidate fixes (Section 7.6): solve the H - ψ system *coupled* rather than alternating; treat the vertical $BB/BH/BL$ coupling *implicitly* (tridiagonal in k); relax the first H update.
2. **Run the SH BALP pass** (`balp.py`) once Pass C is stable, inlining the same $n \leq 1$ mode projection.
3. **Quantitative validation** of the SH port against the Wu Fortran benchmark (Figure 4.10), once it converges.

Appendix A

Notation Glossary

Symbol	Units	Description
a	m	Earth radius 6.371×10^6
α	–	Picard under-relaxation parameter
$\beta = 2\Omega \cos \phi/a$	$\text{s}^{-1} \text{m}^{-1}$	meridional grad. of f
$\bar{\Phi}$	m^2/s^2	climatological mean geopotential
$\bar{\psi}$	m^2/s	climatological mean streamfunction
$\bar{q}$	PVU	climatological mean Ertel PV
$\bar{\zeta} = \nabla^2 \bar{\psi}$	s^{-1}	mean relative vorticity
c_p	J/kg/K	specific heat at const. pressure (1004)
$\delta\Phi = \Phi - \bar{\Phi}$	m^2/s^2	geopotential anomaly
$\delta\psi = \psi - \bar{\psi}$	m^2/s	streamfunction anomaly
$q' = q - \bar{q}$	PVU	Ertel PV anomaly
D_1, D_2	–	1st/2nd derivative matrices in π
$f = 2\Omega \sin \phi$	s^{-1}	Coriolis parameter
$f_0 = f(45^\circ)$	s^{-1}	reference Coriolis
g	m/s^2	gravity (9.80665)
$J(a, b)$	varies	Jacobian
$\kappa = R_a/c_p$	–	0.28586
∇^2, ∇^{-2}	m^{-2}	spherical Laplacian and its inverse
$\lambda_l = l(l+1)/a^2$	m^{-2}	SH Laplacian eigenvalue
Ω	rad/s	Earth rotation rate 7.292×10^{-5}
$\pi = (p/p_0)^\kappa$	–	Exner pressure
p_0	Pa	reference pressure 10^5
ϕ, λ	rad	latitude, longitude
ψ, Φ	$\text{m}^2/\text{s}, \text{m}^2/\text{s}^2$	streamfunction, geopotential
PVU	$10^{-6} \text{K m}^2/\text{kg/s}$	potential vorticity unit
Q, q	PVU	Ertel potential vorticity
$\theta = T(p_0/p)^\kappa$	K	potential temperature
$\zeta = \nabla^2 \psi$	s^{-1}	relative vorticity

Appendix B

Public-API Reference

B.1 Part I — Wu Fortran reference (wu/)

B.1.1 config.py (shared)

- Grid: NX=87, NY=51, NW=10; PLEVS (10 levels, 1000–200 hPa).
- Solver params: OMEGS, OMEGH, PART, INLIN, MAXT, THRSH, piece definitions.
- Paths: WU_IN_DIR, WU_OUT_DIR, FORT_DIR, FIG_DIR.

B.1.2 Fortran programs (wu/fortran/)

- `pvpialln_94UV.f` — Pass A/B: Ertel PV + balanced ψ, Φ (Section 3.2).
- `qinvert21_94.f` (BALNC) — Pass C: total balanced inversion (eqs. (7.5) and (7.6)).
- `qinvertp21_94.f` (BALP) — Pass D: 3-piece perturbation inversion.

B.1.3 Step drivers (wu/steps/)

- `05_wu_pass_ab/` — compile + Pass A/B; writes `meanq, meanh, event_{q,h}.out`.
- `06_wu_pass_c/` — Pass C; writes `event_bal.out`.
- `07_wu_pass_d/` — Pass D; writes `event_pert.out`.
- `08_parse_outputs/` — ERA5 Ertel PV (eq. (4.1)) + Wu/ERA5 comparison.
- `09_pv_advection/` — PV advection by induced winds (eq. (4.2)); writes `pv_advection.nc`.
- `10_fig8/` — Davis 2022 Fig. 8 replica (`fig8_replica.{png,pdf}`).

B.2 Part II — Wu Python finite-difference port (wu_python/)

B.2.1 wu_python/core/grid.py

- FC — Coriolis f (NY,), A — 5-pt Laplacian coeffs (NY,5)
- AP — $\cos \phi$ (NY,), DP/DL — grid spacings [m]
- LON2D, LAT2D — 2-D meshgrid for plotting

B.2.2 `wu_python/core/nondim.py`

- Non-dimensional constants: G, CP, P0, KAP, DPI, FF, TH0, LL
- `PI_WU` — Wu-scaled Exner function $\Pi_{Wu} = C_p(p/p_0)^\kappa/\text{DPI}$
- BB, BH, BL — vertical tridiagonal coefficients (built from `PI_WU`)

B.2.3 `wu_python/core/fd_ops.py`

- `laplacian_5pt(psi)` — 5-point FD Laplacian (5.1)
- `relative_vorticity(U, V)` — $\zeta = \partial_x v - \partial_y u$

B.2.4 `wu_python/core/sor_solver.py`

- `sor_poisson_2d(rhs, bc, omega, max_iter, tol)` — Red-black SOR for $\nabla^2 \psi = \zeta$ (5.2)
- Numba-accelerated with `parallel=True`, thread-safe per-row error reduction

B.2.5 `wu_python/core/pv_calc.py`

- `compute_ertel_pv(U, V, TH, H, FC)` — Ertel PV on σ -levels (6.1)
- `invert_vorticity_balanced(VOR, H, U, V)` — $\nabla^2 \psi = \zeta$ with wind-integrated BC (6.3)
- `_boundary_psi_from_winds(H, U, V)` — Davis eq. 2.40 boundary integration

B.2.6 `wu_python/core/balance.py` (BALNC)

- `balnc_total(Q, H_init, PSI_init, THA_K, ...)` — Coupled ψ - H SOR total-balance inversion (Pass C)
- `_nondim(H, PSI, Q)` — non-dimensionalization: $H_{nd} = H/\text{HND}$, $\psi_{nd} = \psi/(10^5 \cdot \text{HND})$, $q_{nd} = \text{PIF}_k \cdot q/(10^2 \cdot \text{QCONST})$
- `_h_solve_column_parallel` (in `balance_parallel.py`) — Implicit vertical tridiagonal Thomas solver per column, numba `prange` over j

B.2.7 `wu_python/core/piecewise.py` (BALP)

- `balp_pieces(Q_event, Q_mean, PSI_mean, H_mean, ...)` — Piecewise perturbation inversion (Pass D)
- Exact 1:1 port of `qinvertp21_94.f` subroutine BALP
- `_compute_mean_coeffs(HB_nd, SBR_nd)` — pre-compute SLL, SPP, AVO, STB, ASI, BSI, APhi from non-dimensional mean state
- `_balp_solve(SP, HP, QP, TP, coeffs, ...)` — core BALP solver: ψ' SOR (numba 2-D per level) + H' SOR (3-D pointwise)

B.2.8 `wu_python/config.py`

- Solver params: OMEGS, OMEGH, PART, THRSH, MAXT, MAX_ITER
- Parallelism: `N_WORKERS=32` (ProcessPoolExecutor), `NUM_CORES=64` (numba threads)
- Piece definitions: `PIECES` (lower/middle/upper)

B.2.9 Step drivers (`wu_python/steps/`)

- `05_pass_ab/run_pass_ab.py` — Pass A/B: Ertel PV + ψ
- `06_pass_c/run_pass_c.py` — Pass C: total balanced inversion
- `07_pass_d/run_pass_d.py` — Pass D: 3-piece perturbation (`ProcessPoolExecutor`)
- `08_parse_outputs/parse_and_pv.py` — ERA5 vs Wu PV comparison
- `09_pv_advection/pv_advection.py` — PV advection by induced winds
- `10_fig8/fig8_replica.py` — Davis 2022 Fig. 8 replica

B.3 Shared utilities (`pvtend.sh_ops`)

All operate level-by-level on a NH grid via internal parity-mirroring. Retained for reference; the Wu Python track uses finite differences instead.

- `invert_helmholtz_sh`[†], `filter_low_modes_sh`[†] — added for the earlier SH BALNC port; purely additive, backward-compatible.

[†] new functions; no existing `sh_ops` routine was modified.

Bibliography

- [1] A. H. Baker, E. R. Jessup, and T. Manteuffel. A technique for accelerating the convergence of restarted gmres. *SIAM J. Matrix Anal. Appl.*, 26:962–984, 2005.
- [2] B. J. Hoskins, M. E. McIntyre, and A. W. Robertson. On the use and significance of isentropic potential vorticity maps. *Quarterly Journal of the Royal Meteorological Society*, 111(470):877–946, 1985.
- [3] Y. Saad and M. H. Schultz. Gmres: A generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM J. Sci. Stat. Comput.*, 7:856–869, 1986.